

**Detecting Network Intrusions With an Autoencoder: Unsupervised Anomaly Detection on
NSL-KDD**

Ruthvik Nath Bandari

Northeastern University

AAI6680: AI for Cybersecurity

Mimoza Dimodugno, PhD

September 28, 2026

Detecting Network Intrusions With an Autoencoder: Unsupervised Anomaly Detection on NSL-KDD

Signature-based intrusion detection only catches attacks that someone has already described. This lab tested the alternative: an autoencoder trained on normal network connections only, which flags any connection it cannot reconstruct. The official NSL-KDD test set contains 17 attack types that never appear in training, so it also measures whether the detector generalizes to attacks it has never seen.

Method

NSL-KDD (Tavallae et al., 2009) provides 125,973 training and 22,544 test connections, each described by 41 features. Following the brief, I kept numerical features only. I dropped the three categorical fields and the constant *num_outbound_cmds*, which left 37 inputs. The 67,343 normal training records were split 80/20 into training and validation sets. Heavy-tailed count and byte features were log-transformed, and a StandardScaler was then fitted on the training portion only, to prevent leakage.

The autoencoder (37–20–10–20–37, ReLU hidden layers, linear output) was trained with Adam and mean squared error for 50 epochs with early stopping. The anomaly threshold was the 95th percentile of training reconstruction errors. An Isolation Forest (Liu et al., 2008) received identical inputs and the same threshold rule.

Results

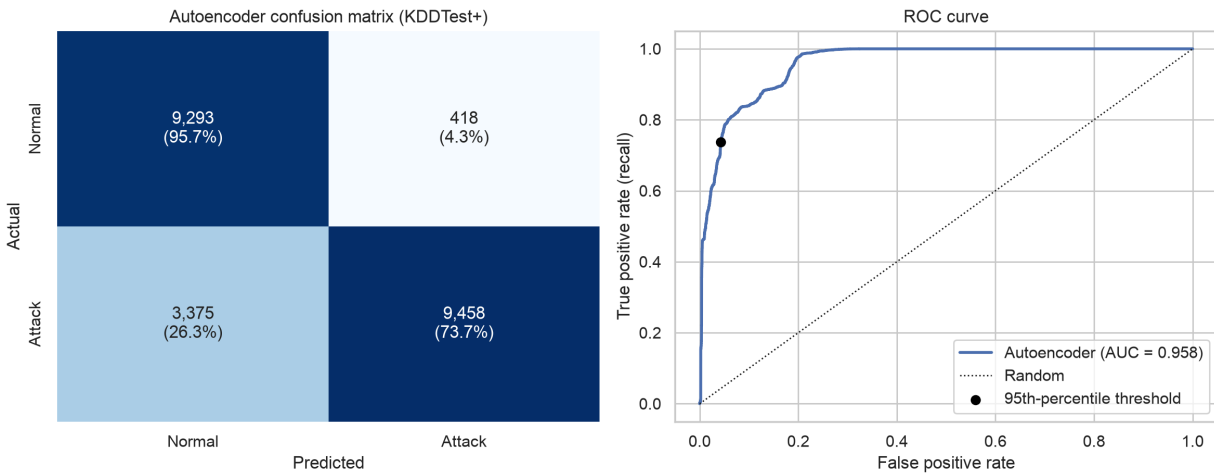
Training loss fell from 0.786 to 0.135, and validation loss tracked it without diverging. The threshold flagged 5.05% of unseen normal validation records, as designed. On the test set, the autoencoder achieved a precision of 0.958, recall of 0.737, F1 of 0.833, and ROC-AUC of 0.958, with a 4.3% false-positive rate (Figure 1).

The autoencoder outperformed the Isolation Forest overall (Table 1), but not everywhere. At a 1% false-positive budget, the Isolation Forest caught 56% of attacks, compared with 48% for the autoencoder. The larger difference was in which attacks each model caught: the autoencoder detected far more remote-to-local (R2L) and user-to-root (U2R) attacks, while the

Isolation Forest was better on scans. The autoencoder also caught 66.5% of the attack types that appear only in the test set.

Figure 1

Autoencoder Confusion Matrix and ROC Curve on the NSL-KDD Test Set



Note. Threshold = 95th percentile of training reconstruction error. The black dot marks this operating point on the ROC curve. Test set: 9,711 normal and 12,833 attack connections.

Table 1*Autoencoder Compared With Isolation Forest on the NSL-KDD Test Set*

Metric	Autoencoder	Isolation Forest
Precision	0.958	0.972
Recall	0.737	0.660
F1-score	0.833	0.786
False-positive rate	0.043	0.025
ROC-AUC	0.958	0.939
Recall at 1% false-positive rate	0.481	0.561
Recall: DoS / Probe	0.856 / 0.814	0.801 / 0.914
Recall: R2L / U2R	0.352 / 0.690	0.067 / 0.500

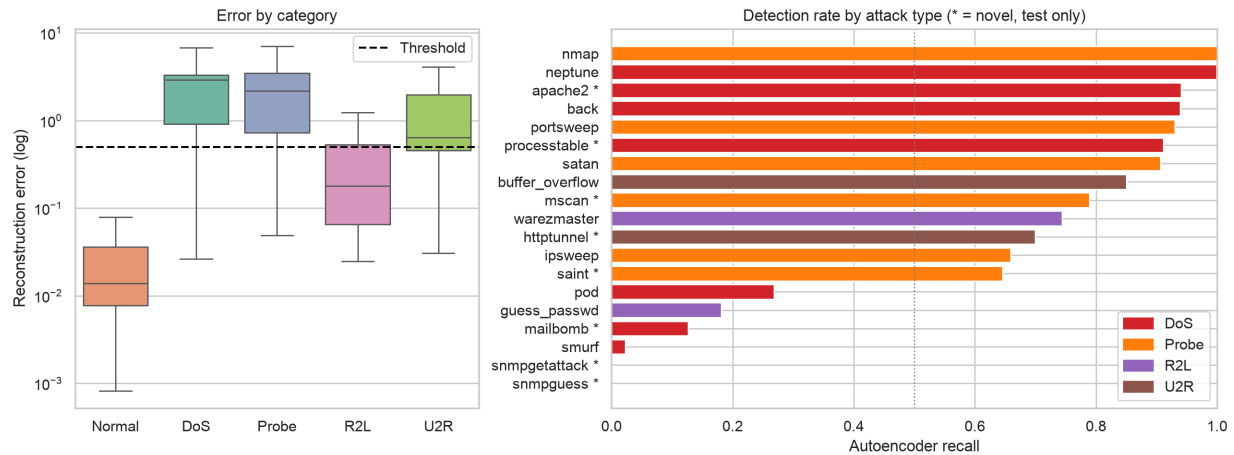
Note. Both models were trained on the same normal-only records and thresholded at the 95th percentile of their own training scores. The better value in each row is in bold.

Attack Type Analysis

The per-attack analysis (Figure 2) shows where detection fails. Floods and scans such as *neptune*, *nmap*, and *satan* exceeded 90% recall. R2L was the hardest category, at 35%. The model never detected *snmpgetattack* or *snmpguess*, and 30% of *snmpgetattack* records are numerically identical to normal records, so no detector using these features could separate them. *guess_passwd* reached only 18% because each guess looks like an ordinary short session. *smurf* (2%) revealed a cost of the numerical-only design: its 1,008-byte packets are anomalous only for ICMP traffic, and the protocol field had been removed. An ablation showed that the log transform raised ROC-AUC from 0.936 to 0.958, so preprocessing is itself a detection decision.

Figure 2

Reconstruction Error by Category and Detection Rate by Attack Type



Note. Left: error distributions on a log scale; the dashed line is the detection threshold. Right: autoencoder recall for attack types with at least 20 test records; an asterisk marks types absent from training.

Reflection

In Module 1, my Gaussian mixture detector missed SSH brute force and botnet traffic because each individual flow looked normal. The autoencoder is a far more flexible model, yet it failed on *guess_passwd* for the same reason. That repetition was the most useful lesson of this lab: model capacity cannot recover information the features do not contain. A deeper network would not catch a password-guessing campaign, but a feature that counts failed logins per source over ten minutes would.

The lab also reinforced that the threshold is a security decision, not a technical detail. Moving from the 95th to the 99th percentile cuts false alarms from 4.3% to under 1%, but drops recall from 74% to 47%. That trade-off belongs to the team answering the alerts. The work I want to do is build detectors that model behavior over time, per user and per host, and pair them with analysts who can act on ranked, explained alerts. This lab showed me that an autoencoder is a useful part of that system, but not the whole system.

References

- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE.
<https://doi.org/10.1109/ICDM.2008.17>
- Tavallae, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (pp. 1–6). IEEE.
<https://doi.org/10.1109/CISDA.2009.5356528>